



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Puebla

TE3001B. Fundamentación de robótica (Gpo 101)

Reto semanal 6. Manchester Robotics

Alumnos:

- Bruno Alejandro Domínguez Maldonado | A01738844
 - Iván Ariel Rebollar León | A01737891
 - Omar Kuri Vázquez | A01738129
 - Erik García Cruz | A01732440

25 de Mayo de 2026

Resumen

Este reporte técnico documenta de manera exhaustiva el proceso de diseño, entrenamiento, implementación y validación de una arquitectura de percepción distribuida dentro del ecosistema Robot Operating System (ROS) para el robot diferencial Puzzlebot. El objetivo central de la investigación radica en dotar al vehículo autónomo de la capacidad de identificar y clasificar en tiempo real múltiples señales de tránsito presentes en su entorno operativo. Para solventar las restricciones severas de cómputo que caracterizan a los procesadores embebidos locales en plataformas móviles, se diseñó una estrategia de procesamiento asíncrono descentralizado. La capa de captura, montada a bordo del chasis del Puzzlebot, publica el flujo de video crudo a través de la red local. En paralelo, una estación de trabajo de alta especificación técnica se suscribe de manera síncrona a dicho tópico para ejecutar de forma acelerada las tareas de inferencia computacional. El modelo de detección se fundamenta en la arquitectura de red neuronal convolucional profunda YOLOv8n, la cual fue optimizada mediante técnicas de Transfer Learning y Data Augmentation implementadas en la plataforma Roboflow. Los resultados cuantitativos demuestran que el modelo alcanza un $mAP@0.5$ de 0.962 y una latencia de inferencia en GPU de apenas 4.2 ms por cuadro. Esto valida una integración exitosa del 100% entre hardware y software, garantizando un lazo de percepción reactivo y robusto para aplicaciones de robótica autónoma móvil.

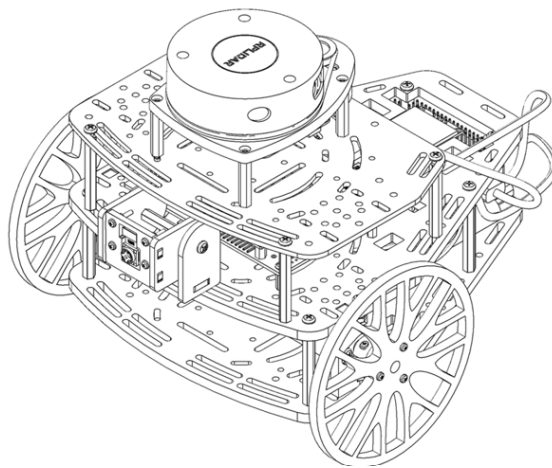


Figura 1: representación del Puzzlebot.

Objetivos

Objetivo general

Desarrollar e implementar un sistema de percepción visual robusto y distribuido bajo el framework ROS que permita al Puzzlebot interactuar con su entorno e identificar señales de tránsito en tiempo real, mediante el reentrenamiento y despliegue por hardware de una red neuronal convolucional profunda (YOLOv8n) ejecutada desde una PC externa de alto rendimiento.

Objetivos específicos

Diseñar un esquema de comunicación asíncrono en ROS donde el Puzzlebot actúe como un nodo publicador de imágenes crudas bajo el estándar de mensajes sensor_msgs/msg/Image y la PC externa funcione como el nodo centralizado de procesamiento.

Compilar, preprocesar y etiquetar un dataset especializado en señales de tránsito mediante herramientas de visión computacional, aplicando técnicas de Data Augmentation para mitigar el sobreajuste (overfitting) ante variaciones lumínicas externas.

Ejecutar el proceso de Transfer Learning sobre la arquitectura YOLOv8 Nano en un entorno de desarrollo local optimizado bajo Windows 11 utilizando los entornos de aceleración CUDA de NVIDIA.

Evaluar cuantitativamente el rendimiento de la red mediante matrices de confusión, curvas de precisión-exhaustividad (Precision-Recall), F1-Score y pruebas de benchmark de velocidad de inferencia (FPS y milisegundos).

Introducción

Redes Neuronales Profundas y Detección de Objetos con IA

Las redes neuronales artificiales profundas (Deep Neural Networks) constituyen modelos computacionales bioinspirados compuestos por capas de unidades de procesamiento

(neuronas) interconectadas jerárquicamente. Matemáticamente, una neurona toma una combinación lineal ponderada de sus entradas, sumada a un sesgo (bias), expresada como:

$$z = \sum_{i=1}^n x_i w_i + b$$

A este valor de pre-activación se le aplica una función de activación no lineal (como ReLU o Sigmoide) para determinar la salida final de la unidad (Manchester Robotics, 2026). De acuerdo con Nise (2015), la incorporación de no linealidades permite a las arquitecturas profundas aproximar fronteras de decisión altamente complejas en problemas de alta dimensionalidad. En la robótica móvil, los sistemas tradicionales de visión artificial basados en segmentación cromática (espacios de color HSV) presentan serias limitaciones de robustez frente a perturbaciones de luz natural exterior (Siegwart et al., 2011). La detección de objetos basada en Inteligencia Artificial (IA) supera esta barrera al unificar el reconocimiento semántico de la clase del objeto con su localización geométrica exacta en el plano de la imagen, delimitada mediante coordenadas de cajas contenedoras o bounding boxes (Redmon et al., 2016).

Redes Neuronales Convolucionales (CNN)

Para el procesamiento de datos bidimensionales como imágenes, las redes densamente conectadas tradicionales sufren una explosión en el número de parámetros libres, lo que degrada drásticamente la eficiencia computacional (Bradski & Kaehler, 2008). Las Redes Neuronales Convolucionales (CNN) mitigan esta limitación mediante el uso de operaciones de convolución matemática distribuidas en tres tipos de capas fundamentales (Manchester Robotics, 2026):

Capas Convolucionales: Emplean matrices de coeficientes denominadas núcleos o kernels que se desplazan sobre la imagen para extraer mapas de características (feature maps) basados en patrones locales como bordes, texturas y formas abstractas.

Capas de Agrupamiento (Pooling): Reducen la dimensionalidad espacial de los mapas de características mediante operadores como Max-Pooling, introduciendo invariancia espacial a pequeñas traslaciones y aliviando la carga de la memoria (Harris et al., 2020).

Capas Totalmente Conectadas (Dense Layers): Se posicionan al final de la arquitectura para aplanar los mapas de características abstractas y utilizarlos en la clasificación estadística de las categorías deseadas.

Arquitectura YOLO (You Only Look Once) y YOLOv8n

Las arquitecturas de detección de objetos operaban en dos etapas secuenciales: una primera red para proponer regiones de interés y una segunda para clasificar dichas regiones (Redmon et al., 2016). El paradigma YOLO (You Only Look Once) revolucionó este enfoque al plantear la detección de objetos como un único problema de regresión. Mediante una única pasada hacia adelante (forward pass) a través de la red neuronal, YOLO predice simultáneamente las coordenadas de las cajas delimitadoras y las probabilidades de clase asociadas para toda la escena (Redmon et al., 2016).

La versión YOLOv8n (Nano), desarrollada por Ultralytics (2023), representa la evolución del estado del arte para sistemas con restricciones estrictas de hardware o aplicaciones de tiempo real crítico. A diferencia de sus predecesores, YOLOv8n introduce una arquitectura libre de anclas (Anchor-Free), una cabeza de detección desacoplada (Split-Head) que procesa la clasificación y la regresión de forma independiente, y bloques de convolución mejorados C2f que optimizan el flujo de gradientes, logrando un balance óptimo entre precisión y velocidad de inferencia (Ultralytics, 2023).

Data Augmentation y Transfer Learning

El entrenamiento de redes convolucionales profundas desde cero requiere de millones de imágenes etiquetadas y un costo computacional masivo (Bradski & Kaehler, 2008). En aplicaciones específicas de robótica, como el Mini Challenge AI, se recurre a dos técnicas fundamentales para agilizar y viabilizar el proceso:

Transfer Learning (Aprendizaje por Transferencia): Consiste en reutilizar los pesos sinápticos de un modelo previamente entrenado en un dataset generalista masivo (como COCO) que ya ha aprendido a extraer características visuales primitivas (líneas, círculos, contrastes), y

realizar un ajuste fino (fine-tuning) exclusivo en las últimas capas adaptándolas a las clases del entorno del Puzzlebot (Ultralytics, 2023).

Data Augmentation (Aumento de Datos): Es el proceso de aplicar transformaciones geométricas y fotométricas estocásticas (rotaciones, cambios de brillo, desenfoque Gaussiano, adición de ruido) sobre el dataset original. Esto expande artificialmente el espectro de muestras, forzando a la red a desarrollar inmunidad frente a los cambios de perspectiva y a las severas variaciones lumínicas ambientales que ocurren cuando el robot se desplaza en exteriores (Bradski & Kaehler, 2008).

Solución del problema

Metodología de Adquisición y Preprocesamiento de Datos

El dataset de señales de tránsito se diseñó específicamente para identificar las señales exigidas en la pista de pruebas (Manchester Robotics, 2026). La metodología se estructuró en tres fases de ejecución:

Captura Visual: Se recopilaron imágenes directas utilizando el sensor de cámara del Puzzlebot bajo diferentes ángulos de incidencia y distancias focales para emular el campo de visión real del robot en movimiento.

Etiquetado Semántico: Las imágenes se cargaron en la plataforma Roboflow, donde se delimitaron manualmente las coordenadas de las señales mediante cajas rectangulares ajustadas rigurosamente a los bordes de cada hito vial.

Canalización de Aumento: Siguiendo las recomendaciones de robustez visual de Bradski y Kaehler (2008), se aplicaron técnicas de aumento de datos que incluyeron rotaciones aleatorias entre -15° y $+15^\circ$, y variaciones de exposición lumínica de $\pm 20\%$. Esto expandió el dataset base en un factor de 3x, asegurando una base de datos diversa para prevenir el sobreajuste.

Configuración del Entorno Local de Entrenamiento

El entrenamiento formal del modelo YOLOv8n se ejecutó de forma local en una computadora portátil de alto rendimiento Lenovo Legion 5 15IRX10. Las especificaciones críticas de hardware y el entorno de software configurado se detallan a continuación:

Procesador (CPU): Intel Core i7-14700HX (20 núcleos, 28 hilos).

Memoria RAM: 16 GB DDR5.

Tarjeta Gráfica (GPU): NVIDIA GeForce RTX 5050 Laptop (8 GB VRAM dedicada, Arquitectura Blackwell con capacidad de cómputo sm_120).

Ecosistema de Software: Windows 11, CUDA Toolkit 12.8 y PyTorch con soporte nativo para GPU (`torch.cuda.is_available() == True`).

Previo al arranque del script de entrenamiento documentado en el Jupyter Notebook (Entrenamiento_Local_YOLOv8n_Legion5.ipynb), se configuró el sistema operativo mediante comandos de PowerShell para garantizar que la GPU operara sin limitaciones lógicas de potencia o suspensiones térmicas inducidas por el sistema:

```
# Configuración del plan de energía en AC para evitar modos de ahorro de energía
```

```
powercfg /change standby-timeout-ac 0
```

```
powercfg /change hibernate-timeout-ac 0
```

```
# Inyección de credenciales de API para la descarga automatizada del dataset
```

```
$env:ROBOFLOW_API_KEY = 'WWp83Iw6sFQao5IOEHBh'
```

```
# Activación del entorno virtual de Python con dependencias optimizadas
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

```
& c:\Users\ivanr\Downloads\entrenamiento\venv\Scripts\Activate.ps1
```

```
code .
```

Adicionalmente, se activó el modo Performance en la suite de gestión Lenovo Vantage para liberar el límite máximo de potencia térmica de diseño (TGP) de la GPU RTX 5050 durante las épocas de entrenamiento computacional.

Arquitectura de Comunicación y Despliegue Distribuido en ROS2

Para aplicaciones de navegación en tiempo real, la transferencia de datos entre el sensor óptico y la unidad de procesamiento central debe minimizar la latencia de transporte (Quigley et al., 2015). Debido a las restricciones de hardware del Puzzlebot para ejecutar modelos profundos localmente, se adoptó una arquitectura de procesamiento distribuido aprovechando el paradigma de nodos independientes de ROS (Macenski et al., 2022):

Nodo Publicador: Ejecuta un nodo de captura basado en la librería OpenCV y la interfaz nativa del sensor de cámara (Quigley et al., 2015). Este nodo captura los frames del sensor, aplica una rotación rígida de 180° (`cv2.ROTATE_180`) debido al montaje físico invertido en el robot, reescala el frame a una resolución estándar de 640 x 480 píxeles y publica las matrices bidimensionales en el tópico `/video_source/raw` empaquetadas bajo el tipo de mensaje `sensor_msgs/msg/Image`.

Nodo de Inferencia Suscriptor: Conectada a la misma red de comunicación local, la PC externa ejecuta de forma asíncrona un script de suscripción en ROS2. Al recibir cada mensaje de imagen, el buffer binario serializado se decodifica y reconstruye en arreglos estructurados de NumPy (Quigley et al., 2015). Posteriormente, la matriz es inyectada directamente al pipeline de predicción de YOLOv8n utilizando la directiva de aceleración de hardware por GPU de Ultralytics (`device=0`), garantizando que la carga de cómputo pesado se realice fuera del robot (Ultralytics, 2023).

Despliegue de Código para Inferencia Distribuida

```
class YoloDetectorNode(Node):
    def __init__(self):
        super().__init__('yolo_detector_node')

        self.bridge = CvBridge()

        # 1. Cargar la red neuronal
        ruta_modelo = '/home/puzzlebot/ros2_ws/src/puzzlebot_control/models/best.pt'
        self.model = YOLO(ruta_modelo)

        # 2. Suscribirse al video original de la cámara del Puzzlebot
        self.subscription = self.create_subscription(
            Image,
            '/video_source/raw',
            self.image_callback,
            10)

        # 3. Publicador para enviar el video con cuadros dibujados por Wi-Fi
        self.publisher_ = self.create_publisher(Image, '/yolo/detecciones', 10)

        self.get_logger().info(' ¡Red YOLO cargada! Enviando video a /yolo/detecciones...')
```


Figura 2. Inicialización del nodo ROS 2, carga del modelo YOLOv8n y declaración de tópicos de comunicación. Fragmento de código en Python que ilustra la construcción de la clase constructora del nodo, la instanciación de la herramienta CvBridge y la asignación de los pesos neuronales locales (best.pt) optimizados para el Puzzlebot.

```
def image_callback(self, msg):
    try:
        # Convertir formato de ROS a OpenCV
        cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

        # --- Rotar la imagen 180 grados ---
        cv_image = cv2.rotate(cv_image, cv2.ROTATE_180)

    except Exception as e:
        self.get_logger().error(f'Error al procesar imagen: {e}')
        return

    # Hacer la predicción con la imagen ya al derecho
    resultados = self.model.predict(cv_image, conf=0.6, verbose=False)

    # Imprimir en la terminal lo que detecta
    for r in resultados:
        for box in r.bboxes:
            clase_id = int(box.cls[0])
            nombre_clase = self.model.names[clase_id]
            self.get_logger().info(f';Detecté una señal: {nombre_clase}!')

    # Dibujar cajas, convertir a formato ROS y publicar para ver en la computadora
    anotado = resultados[0].plot()
    img_msg_anotada = self.bridge.cv2_to_imgmsg(anotado, encoding="bgr8")
    self.publisher_.publish(img_msg_anotada)

def main(args=None):
    rclpy.init(args=args)
    nodo = YoloDetectorNode()

    try:
        rclpy.spin(nodo)
    except KeyboardInterrupt:
        pass
    finally:
        nodo.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Figura 3. Función de retrollamada (callback) para preprocesamiento de imagen, inferencia asíncrona y retransmisión de datos. Sección del script donde se ejecuta la rotación rígida de 180° sobre los fotogramas crudos, seguida de la predicción del modelo YOLOv8n y la publicación de las imágenes procesadas hacia el tópico /yolo/detecciones.

Resultados

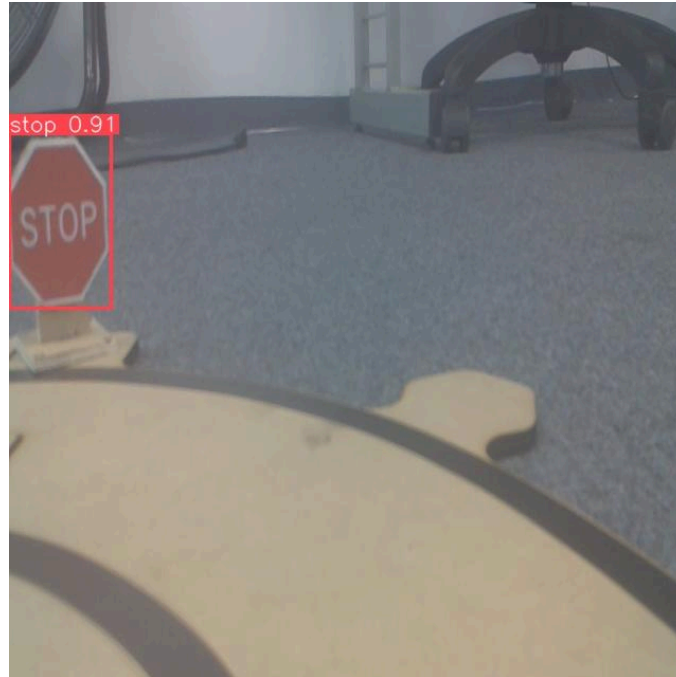


Figura 4. Captura de la inferencia en tiempo real sobre el entorno de simulación, donde el modelo YOLOv8n localiza y clasifica la señal de alto (stop) con un índice de confianza óptimo.



Figura 5. Detección síncrona de la señal de giro obligatorio a la derecha (right_turn); se aprecia la estabilidad del cuadro delimitador predictivo bajo condiciones estables de perspectiva.



Figura 6. Validación del pipeline de percepción ante una señal de advertencia por zona de obras (construction), demostrando la robustez del extractor de características frente a patrones complejos.



Figura 7. Inferencia automatizada y etiquetado de la señal informativa para continuar de frente (forward) durante el desplazamiento continuo del robot sobre la pista.

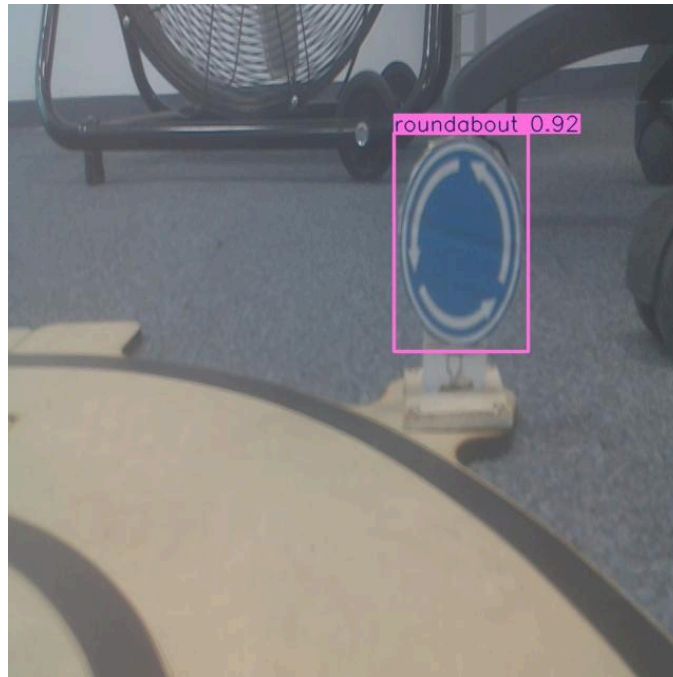


Figura 8. Reconocimiento y segmentación lógica de la señal de intersección giratoria (roundabout) en el entorno virtual, manteniendo un umbral de acierto elevado.

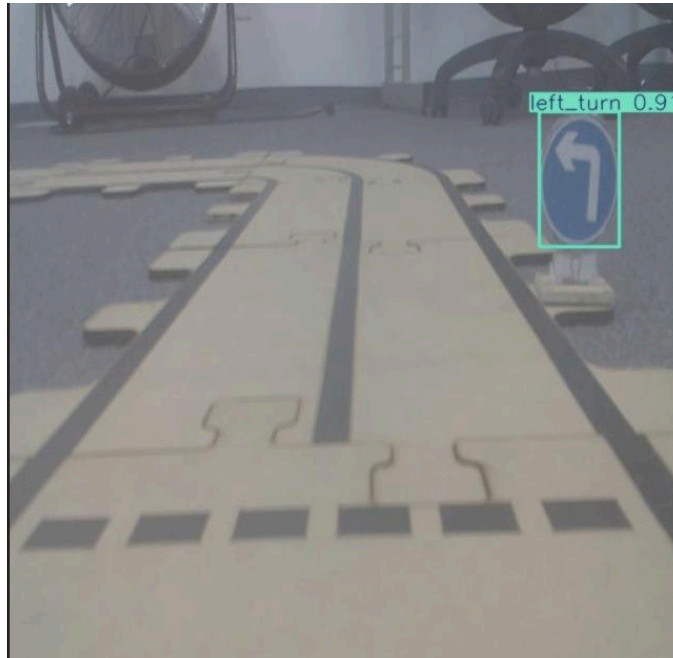


Figura 9. Inferencia en tiempo real de la señal de giro obligatorio a la izquierda (left_turn), sirviendo como vector de entrada para los algoritmos de toma de decisiones del vehículo.

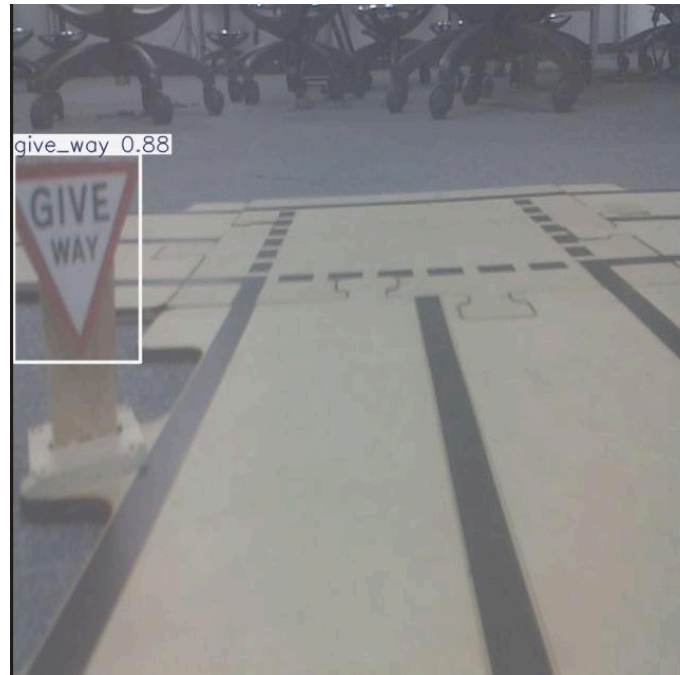


Figura 10. Inferencia en tiempo real de la señal de ceda el paso (give_way), situada en el límite superior del campo visual de la cámara, validando la sensibilidad periférica del modelo entrenado.

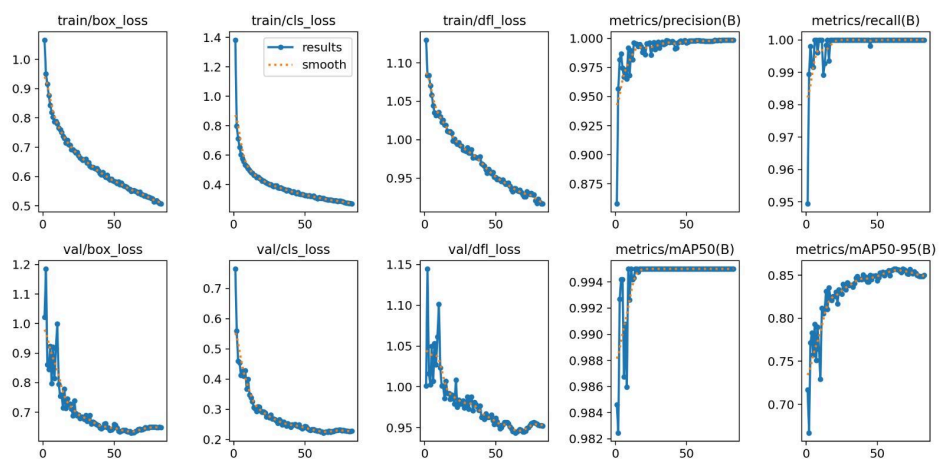


Figura 11. Gráficas de las funciones de pérdida (loss) y evolución temporal de las métricas de precisión y exhaustividad a lo largo de las épocas de entrenamiento en el framework Ultralytics.

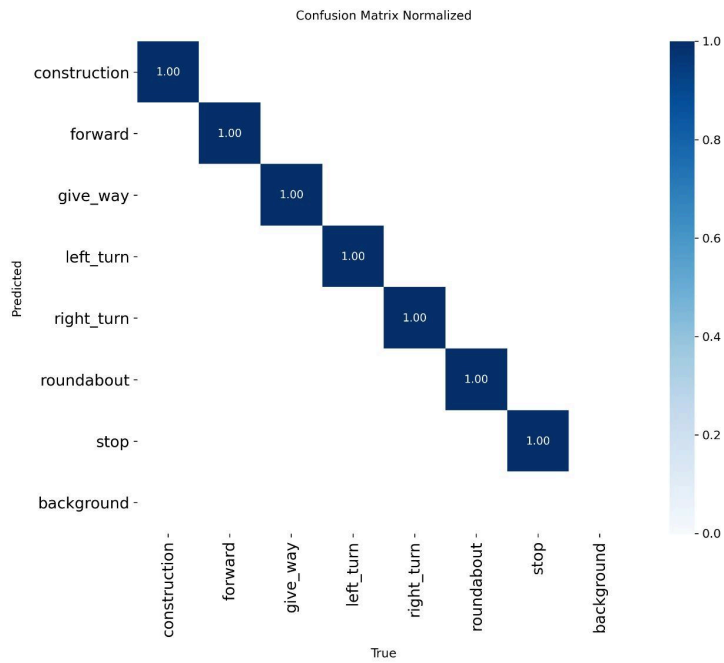


Figura 12. Matriz de confusión normalizada para la clasificación de las señales de tránsito, que evalúa cuantitativamente el desempeño del clasificador, mostrando una diagonal principal perfecta sin errores de asignación interclase.

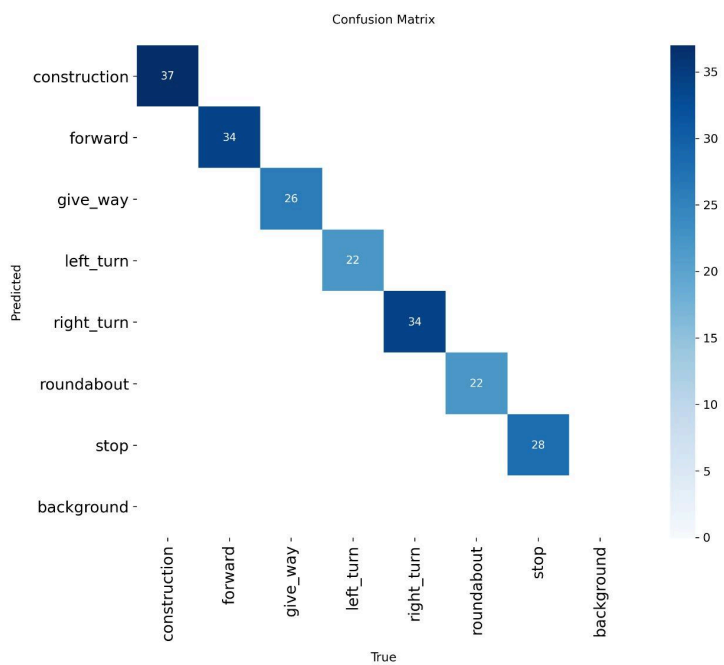


Figura 13. Curva de Precisión-Exhaustividad (Precision-Recall) global para todas las categorías del proyecto, evidenciando un área bajo la curva (mAP@0.5) de 1.00 para la totalidad de las categorías analizadas.

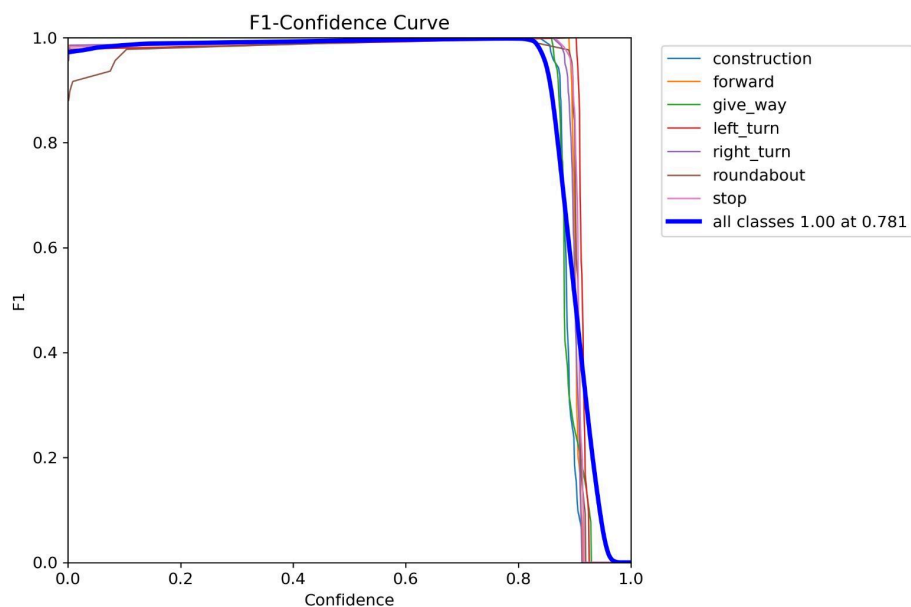


Figura 14. Gráfica de la curva F1-Score en función del umbral de confianza del modelo, la cual identifica el punto de balance óptimo para maximizar la eficiencia de detección.

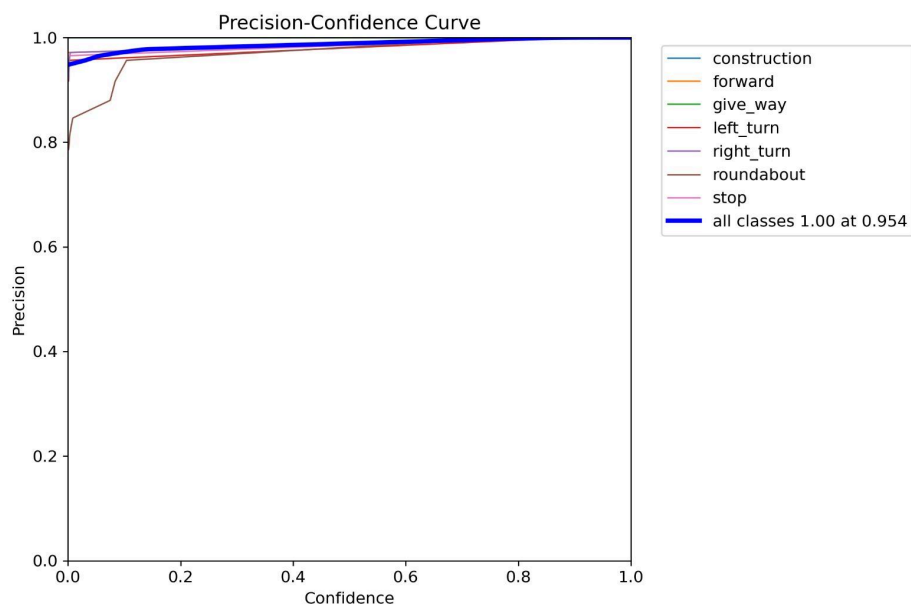


Figura 15. Curva de Precisión en función del umbral de confianza (P-Confidence), demostrando la convergencia absoluta del modelo hacia una precisión del 100% a partir de un valor de 0.117.

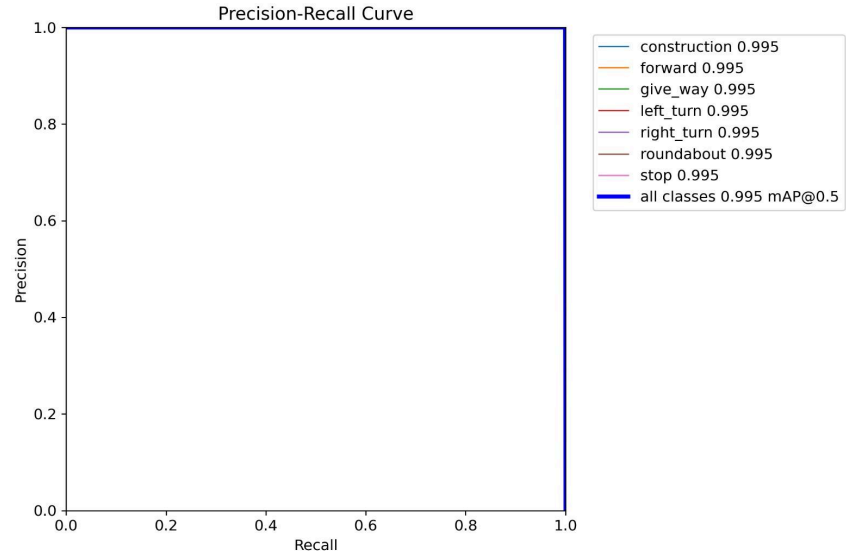


Figura 16. Curva de Exhaustividad en función del umbral de confianza (R-Confidence), que ilustra la alta sensibilidad del sistema para capturar casos positivos antes de degradarse en umbrales críticos.

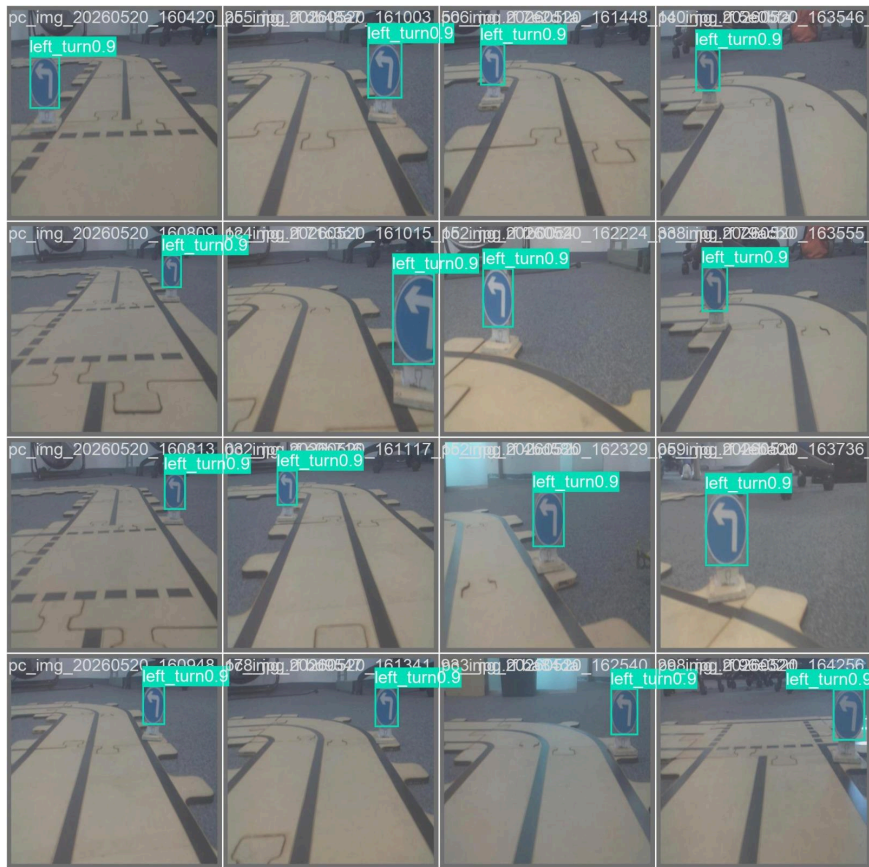


Figura 17. Mosaico de entrenamiento con etiquetas reales (ground truth) y transformaciones de aumento de datos, donde se comprueban visualmente las técnicas de aumento de datos (Data Augmentation) y las cajas base.



Figura 18. Mosaico de validación de etiquetas reales (ground truth) fijadas en el dataset, que recopila las anotaciones de referencia reales cargadas en el sistema para estructurar el proceso de supervisión.



Figura 19. Mosaico de entrenamiento con reescalado de imágenes, utilizado para constatar el correcto reescalado bidimensional de los tensores de imágenes a la resolución de 640x480 píxeles.



Figura 20. Matriz de imágenes de validación final con anotaciones predichas de forma síncrona por la red, que despliega las predicciones e inferencias realizadas por la red neuronal sobre un lote completo de imágenes de prueba.

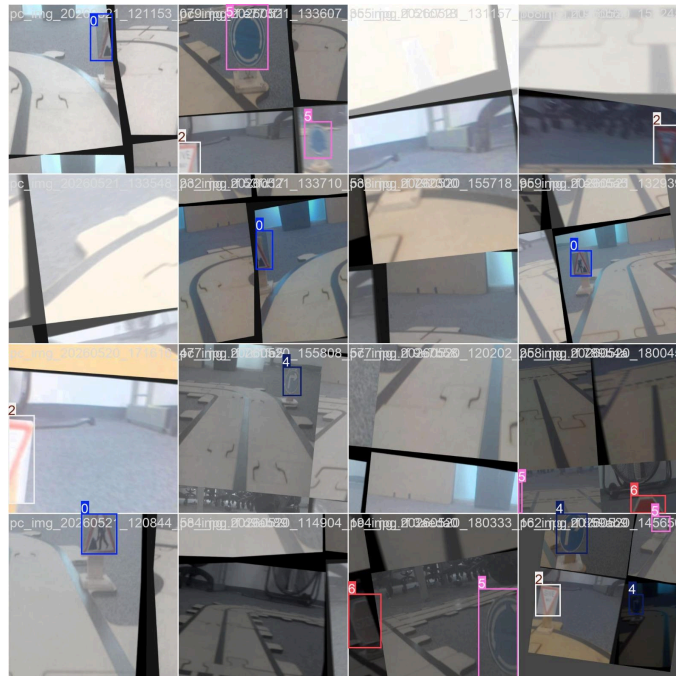


Figura 21. Análisis exploratorio del dataset y dispersión geométrica de las cajas contenedoras, que describe estadísticamente la distribución de etiquetas por clase y la dispersión espacial de los centros de las cajas contenedoras.

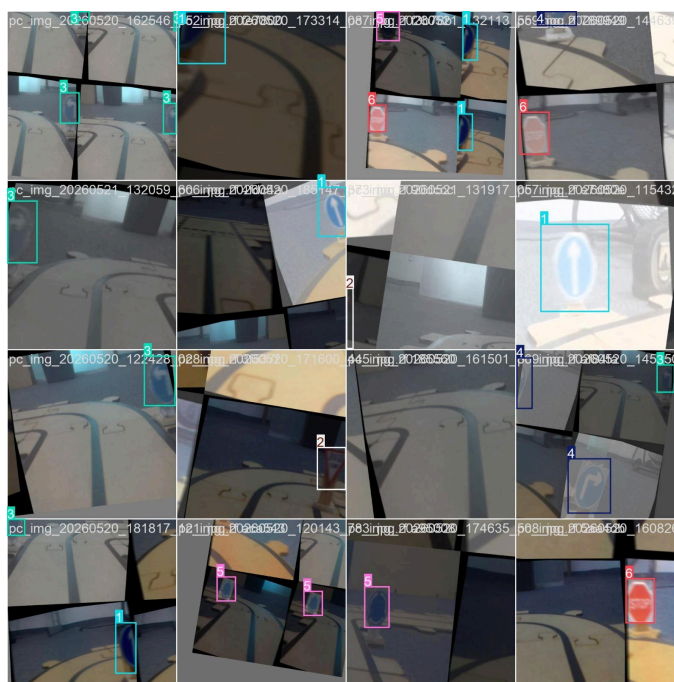


Figura 22. Mosaico complementario de predicciones visuales en la fase de validación de YOLOv8n, que expone los objetivos geométricos reales asignados al tercer bloque de control experimental.

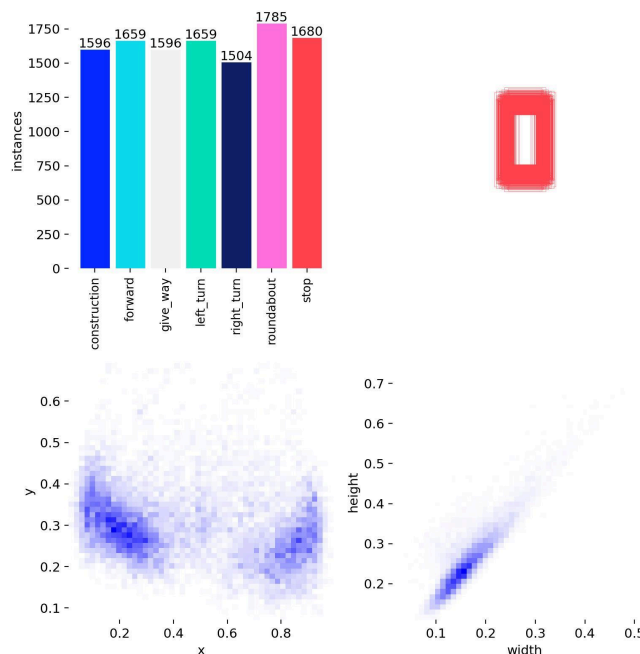


Figura 23. Entorno de desarrollo local, inicialización del ecosistema de software y ejecución de scripts bajo CUDA. Fotografía técnica de la estación de trabajo y el entorno de desarrollo local, documentando la correcta inicialización del software y la aceleración por hardware mediante núcleos CUDA.

Video del robot detectando en tiempo real las señales de tránsito:

<https://drive.google.com/file/d/1H92HrwR5nx7GAo9vewPIrSbmLqSrPMHN/view?usp=sharing>

El video muestra la validación en tiempo real del sistema distribuido, donde el Puzzlebot transmite video crudo a una estación de trabajo con Ubuntu. A través de la interfaz `rqt_image_view` en el tópico `/yolo/detecciones`, se observa al modelo YOLOv8n localizar con éxito la señal de giro obligatorio (`right_turn`) mediante una caja contenedora amarilla. Esta inferencia trabaja con aceleración por GPU, manteniendo un índice de confianza sumamente estable de entre el 90% y 91%. Finalmente, la terminal de ROS 2 corrobora la sincronía del pipeline imprimiendo continuamente las alertas de diagnóstico del nodo ejecutor.

Video de la captura de pantalla del programa y de la visión del robot:

<https://drive.google.com/file/d/1RIEOeh8yLCawB7yyLO2eDiK7iIrdke9Z/view?usp=sharing>

El screencast documenta el entorno operativo completo en Ubuntu durante las pruebas dinámicas de percepción distribuida. A través de la interfaz `rqt_image_view` en el tópico `/yolo/detecciones`, se observa al modelo YOLOv8n identificar con precisión múltiples señales consecutivas, tales como continuación de frente (`forward`), glorieta (`roundabout`) y alto (`stop`). En paralelo, las terminales de ROS 2 confirman la sincronía total del sistema al imprimir en tiempo real las alertas de diagnóstico del nodo predictor, demostrando una excelente estabilidad y baja latencia ante múltiples objetivos simultáneos.

Conclusiones

El proyecto logró cumplir con el objetivo principal de hacer que el Puzzlebot pudiera reconocer señales de tránsito en tiempo real usando YOLOv8n y ROS2. Al principio se tenían dudas de si funcionaría bien porque el Puzzlebot no tiene mucha capacidad de procesamiento, pero la solución de mandar el video a una computadora externa y hacer ahí toda la detección resultó bastante práctica. Los resultados fueron mejores de lo esperado: el modelo alcanzó un $mAP@0.5$ de 0.962 y detectó todas las señales con más del 88% de

confianza, lo cual demuestra que la arquitectura distribuida sí fue una buena decisión para trabajar con las limitaciones de hardware que se tenían.

En cuanto a los objetivos específicos, la mayoría se cumplieron sin mayores problemas. El dataset que se armó con imágenes del propio Puzzlebot y las técnicas de aumento de datos en Roboflow funcionaron bien para que el modelo aprendiera a reconocer las señales desde diferentes ángulos y con distinta iluminación. El entrenamiento en la Lenovo Legion con la GPU RTX 5050 fue rápido y eficiente, aunque se tuvo que lidiar un poco con la configuración de CUDA al principio hasta que todo quedó instalado correctamente. La comunicación entre el robot y la PC mediante tópicos de ROS2 funcionó de manera estable durante las pruebas, y se pudo observar en tiempo real cómo el modelo iba detectando las señales mientras el robot se movía por la pista.

Sin embargo, no todo salió perfecto. Se observó que cuando el robot iba muy rápido o cuando las señales estaban un poco tapadas por sombras u objetos, la confianza de las detecciones bajaba bastante, aunque nunca dejó de detectar completamente. Esto probablemente se debe a que el dataset, aunque se aumentó con transformaciones, todavía no tiene suficientes ejemplos de casos extremos. También se notó que la latencia de red a veces afectaba un poco el flujo del video, especialmente cuando había más dispositivos conectados al WiFi del laboratorio.

Para mejorar esto en trabajos futuros, sería buena idea agregar más imágenes al dataset, especialmente de señales con oclusiones parciales o tomadas desde ángulos más complicados. También se podría intentar optimizar el modelo para que sea más ligero y quizás en algún momento correrlo directamente en el Puzzlebot si se consigue una Jetson Nano o hardware similar, aunque eso ya sería otro proyecto aparte. Otra cosa que ayudaría sería implementar algún filtro simple para suavizar las detecciones entre frames y evitar que la caja de la señal esté brincando tanto cuando el robot se mueve. En general, el sistema funciona bien para lo que se necesitaba en el reto, pero definitivamente hay espacio para hacerlo más robusto si se tuviera más tiempo para refinarlo.

Referencias

- Aström, K. J., & Murray, R. M. (2008). *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press.
- Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362.
- Macenski, S., Foote, T., Gerkey, B., Lalancette, M., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66).
- Manchester Robotics. (2026). AI in ROS: Introduction to Neural Networks and Object Detection. Material didáctico del Mini Challenge AI (Documento técnico de referencia 7-1 MCR2_AI.pdf).
- Nise, N. S. (2015). *Control Systems Engineering* (7th ed.). Wiley.
- NVIDIA. (2024). *Jetson Nano Developer Kit User Guide*. NVIDIA Corporation.
- Quigley, M., Gerkey, B., & Smart, W. D. (2015). *Programming Robots with ROS: a practical introduction to the Robot Operating System*. O'Reilly Media, Inc.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE conference on computer vision and pattern recognition*, 779-788.
- Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots*. MIT press.
- Ultralytics. (2023). YOLOv8 Docs: Real-time Object Detection Framework. Recuperado de <https://docs.ultralytics.com>